

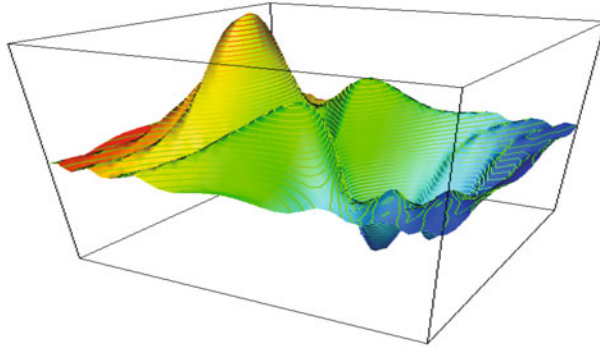


Fig. 2.2. Illustration of Wright's adaptive landscape with two traits

population has a finite size, and random choices are made in the selection and variation operators, it is common to observe the phenomenon of **genetic drift**, whereby highly fit individuals may be lost from the population, or the population may suffer from a loss of variety concerning some traits. This can have the effect that populations 'melt down' the hill, and enter low-fitness valleys. The combined global effects of drift and selection enable populations to move uphill as well as downhill, and of course there is no guarantee that the population will climb back up the same hill. Escaping from locally optimal regions is hereby possible, and according to Wright's shifting balance theory the maximum of a fixed landscape can be reached.

2.3.2 Genetics

The microscopic view of natural evolution is offered by molecular genetics. It sheds light on the processes below the level of visible phenotypic features, in particular relating to heredity. The fundamental observation from genetics is that each individual is a dual entity: its phenotypic properties (outside) are represented at a genotypic level (inside). In other words, an individual's **genotype** encodes its **phenotype**. **Genes** are the functional units of inheritance encoding phenotypic characteristics. For instance, visible properties like the fur colour or tail length could be determined by genes. Here it is important to distinguish genes and **alleles**. An allele is one of the possible values that a gene can have – so its relationship to a gene is just like that of a specific value to a variable in mathematics. To illustrate this by an oversimplified example, bears could have a gene that determines fur colour, and for a polar bear we would expect to see the allele that specifies the colour white. In natural systems the genetic encoding is not one-to-one: one gene might affect more phenotypic traits (**pleiotropy**), and in turn one phenotypic trait can be determined by more than one gene (**polygeny**). Phenotypic variations are

In general, it cannot be claimed that increasing the arity of recombination has a positive effect on the performance of an EA – this depends very much on the type of recombination and the problem at hand. However, systematic studies on landscapes with tuneable ruggedness [143] and a large number of experimental investigations on various problems clearly show that using more than two parents can accelerate evolutionary search and be advantageous in many cases.

4.2 Binary Representation

The first representation we look at is one of the simplest – the binary one used in Sect. 3.3. This is one of the earliest representations, and historically many genetic algorithms (GAs) have (mistakenly) used this representation almost independently of the problem they were trying to solve. Here the genotype consists simply of a string of binary digits – a bit-string.

For a particular application we have to decide how long the string should be, and how we will interpret it to produce a phenotype. In choosing the genotype–phenotype mapping for a specific problem, one has to make sure that the encoding allows that all possible bit strings denote a valid solution to the given problem¹ and that, vice versa, all possible solutions can be represented.

For some problems, particularly those concerning Boolean decision variables, the genotype–phenotype mapping is natural. One example is the knapsack problem described in Sect. 3.4.2, where for each possible item a Boolean decision was evolved, denoting whether it was included in the final solution. Frequently bit-strings are used to encode other nonbinary information. For example, we might interpret a bit-string of length 80 as 10 integers, each encoded as 8-bit integers (allowing for 256 possible values), or five 16-bit real numbers. Using bit-strings to encode nonbinary information is usually a mistake, and better results can be obtained by using the integer or real-valued representations directly.

One of the problems of coding numbers in binary is that different bits have different significance, and so the effect of a single bit mutation is very variable. Using standard binary code has the disadvantage that the Hamming distance between two consecutive integers is often not equal to one. If the goal is to evolve an integer number, you would like to have equal probabilities of changing a 7 into an 8 or a 6. However, changing 0111 to 1000 requires four bit-flips while changing it to 0110 takes just one. Thus with a mutation operator that randomly, and independently, changes each allele value with probability $p_m < 0.5$, the probability of changing 7 to 8 is much less than changing 7 to 6. This can be helped by using **Gray coding**, a variation on the way that integers are mapped on bit strings where consecutive integers always have Hamming distance one.

¹ In practice this restriction to validity is not always possible; see Chap. 13 for a more complete discussion of this issue.

4.2.1 Mutation for Binary Representation

Although a few other schemes have been occasionally used, the most common mutation operator for binary encodings considers each gene separately and allows each bit to flip (i.e., from 1 to 0 or 0 to 1) with a small probability p_m . The actual number of values changed is thus not fixed, but depends on the sequence of random numbers drawn, so for an encoding of length L , on average $L \cdot p_m$ values will be changed. In Fig. 4.1 this is illustrated for the case where the third, fourth, and eighth random values generated are less than the bitwise mutation rate p_m .



Fig. 4.1. Bitwise mutation for binary encodings

A number of studies and recommendations have been made for the choice of suitable values for the bitwise mutation rate p_m . Most binary coded GAs use mutation rates in a range such that on average between one gene per generation and one gene per offspring is mutated. However, it is worth noting at the outset that the most suitable choice to use depends on the desired outcome. For example, does the application require a population in which *all* members have high fitness, or simply that *one* highly fit individual is found? The former suggests a lower mutation rate, less likely to disrupt good solutions. In the latter case one might choose a higher mutation rate if the potential benefits of ensuring good coverage of the search space outweighed the cost of disrupting copies of good solutions².

4.2.2 Recombination for Binary Representation

Three standard forms of recombination are generally used for binary representations. They all start from two parents and create two children, although all of these have been extended to the more general case where a number of parents may be used [152], and there are also situations in which only one of the offspring might be considered (Sect. 5.1).

One-Point Crossover One-point crossover was the original recombination operator proposed in [220] and examined in [102]. It works by choosing a

² In fact this example illustrates that the algorithm's parameters cannot be chosen independently: in the second case we might couple higher mutation rates with a more aggressive selection policy to ensure the best solutions were not lost.

random number r in the range $[1, l - 1]$ (with l the length of the encoding), and then splitting both parents at this point and creating the two children by exchanging the tails (Fig. 4.2, top). Note that by using the range $[1, l - 1]$ the crossover point is prevented from falling before the first position ($r = 0$) or after the last position ($r = l$).

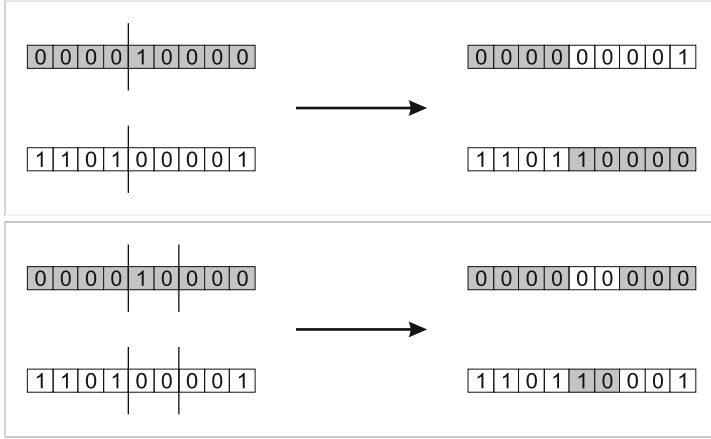


Fig. 4.2. One-point crossover (top) and n -point crossover with $n = 2$ (bottom)

n -Point Crossover One-point crossover can easily be generalised to n -point crossover, where the chromosome is broken into more than two segments of contiguous genes, and the offspring are created by taking alternative segments from the parents. In practice this means choosing n random crossover points in $[1, l - 1]$, which is illustrated in Fig. 4.2 (bottom) for $n = 2$.

Uniform Crossover The previous two operators worked by dividing the parents into a number of sections of contiguous genes and reassembling them to produce offspring. In contrast to this, uniform crossover [422] works by treating each gene independently and making a random choice as to which parent it should be inherited from. This is implemented by generating a string of l random variables from a uniform distribution over $[0, 1]$. In each position, if the value is below a parameter p (usually 0.5), the gene is inherited from the first parent; otherwise from the second. The second offspring is created using the inverse mapping. This is illustrated in Fig. 4.3.

In our discussion so far, we have suggested that in the absence of prior information, recombination worked by randomly mixing parts of the parents. However, as Fig. 4.2 illustrates, n -point crossover has an inherent bias in that it tends to keep together genes that are located close to each other in

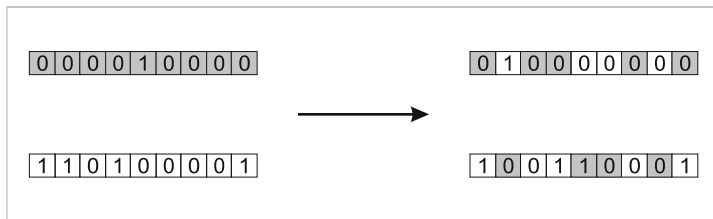


Fig. 4.3. Uniform crossover. The array $[0.3, 0.6, 0.1, 0.4, 0.8, 0.7, 0.3, 0.5, 0.3]$ of random numbers and $p = 0.5$ were used to decide inheritance for this example.

the representation. Furthermore, when n is odd (e.g., one-point crossover), there is a strong bias against keeping together combinations of genes that are located at opposite ends of the representation. These effects are known as **positional bias** and have been extensively studied from both a theoretical and experimental perspective [157, 412] (see Sect. 16.1 for more details). In contrast, uniform crossover does not exhibit any positional bias. However, unlike n -point crossover, uniform crossover does have a strong tendency towards transmitting 50% of the genes from each parent and against transmitting an offspring a large number of coadapted genes from one parent. This is known as **distributional bias**.

The general nature of these algorithms (and the No Free Lunch theorem [467], Sect. 16.10) make it impossible to state that one or the other of these operators performs best on any given problem. Nevertheless, an understanding of the types of bias exhibited by different recombination operators can be invaluable when designing an algorithm for a particular problem, particularly if there are known patterns or dependencies in the chosen representation that can be exploited. To use the knapsack problem as an example, it might make sense to use an operator that is likely to keep together the decisions for the first few heaviest items. If the items are ordered by weight (cost) in our representation, then we could make this more likely by using n -point crossover with its positional bias. However, if we used a random ordering this might actually make it less likely that co-adapted values for certain decisions were transmitted together, so we might prefer uniform crossover.

4.3 Integer Representation

As we hinted in the previous section, binary representations are not always the most suitable if our problem more naturally maps onto a representation where different genes can take one of a set of values. One obvious example of when this might occur is the problem of finding the optimal values for a set of variables that all take integer values. These values might be unrestricted (i.e., any integer value is permissible), or might be restricted to a finite set: for example, if we are trying to evolve a path on a square grid, we might restrict the

values to the set $\{0,1,2,3\}$ representing $\{North, East, South, West\}$. In either case an integer encoding is probably more suitable than a binary encoding. When designing the encoding and variation operators, it is worth considering whether there are any natural relations between the possible values that an attribute can take. This might be obvious for **ordinal attributes** such as integers (2 is more like 3 than it is 389), but for **cardinal attributes** such as the compass points above, there may not be a natural ordering.³

To give a well-known example of where there is no natural ordering, let us consider the graph k -colouring problem. Here we are given a set of points (vertices) and a list of connections between them (edges). The task is to assign one of k colours to each vertex, so that no two vertices which are connected by an edge share the same colour. For this problem there is no natural ordering: ‘red’ is no more like ‘yellow’ than ‘blue’, as long as they are different. In fact, we could assign the colours to the k integers representing them in any order, and still get valid equivalent solutions.

4.3.1 Mutation for Integer Representations

For integer encodings there are two principal forms of mutation used, both of which mutate each gene independently with user-defined probability p_m .

Random Resetting Here the bit-flipping mutation of binary encodings is extended to random resetting: in each position independently, with probability p_m , a new value is chosen at random from the set of permissible values. This is the most suitable operator to use when the genes encode for cardinal attributes, since all other gene values are equally likely to be chosen.

Creep Mutation This scheme was designed for ordinal attributes and works by adding a small (positive or negative) value to each gene with probability p . Usually these values are sampled randomly for each position, from a distribution that is symmetric about zero, and is more likely to generate small changes than large ones. It should be noted that creep mutation requires a number of parameters controlling the distribution from which the random numbers are drawn, and hence the size of the *steps* that mutation takes in the search space. Finding appropriate settings for these parameters may not be easy, and it is sometimes common to use more than one mutation operator in tandem from integer-based problems. For example, in [98] both a “big creep” and a “little creep” operator are used. Alternatively, random resetting might be used with low probability, in conjunction with a creep operator that tended to make small changes relative to the range of permissible values.

³ There are various naming conventions used to distinguish these two types of attributes. These are discussed further in Chap. 7 and displayed in [Table 7.1](#).

4.3.2 Recombination for Integer Representation

For representations where each gene has a finite number of possible allele values (such as integers) it is normal to use the same set of operators as for binary representations. On the one hand, these operators are valid: the offspring would not fall outside the given genotype space. On the other hand, these operators are also sufficient: it usually does not make sense to consider ‘blending’ allele values of this sort. For example, even if genes represent integer values, averaging an even and an odd integer yields a non-integral result.

4.4 Real-Valued or Floating-Point Representation

Often the most sensible way to represent a candidate solution to a problem is to have a string of real values. This occurs when the values that we want to represent as genes come from a continuous rather than a discrete distribution — for example, if they represent physical quantities such as the length, width, height, or weight of some component of a design that can be specified within a tolerance smaller than integer values. A good example would be the satellite dish holder boom described in Sect. 2.4, where the design is encoded as a series of angles and spar lengths. Another example might be if we wished to use an EA to evolve the weights on the connections between the nodes in an artificial neural network. Of course, on a computer the precision of these real values is actually limited by the implementation, so we will refer to them as floating-point numbers. The genotype for a solution with k genes is now a vector $\langle x_1, \dots, x_k \rangle$ with $x_i \in \mathbb{R}$.

4.4.1 Mutation for Real-Valued Representation

For floating-point representations, it is normal to ignore the discretisation imposed by hardware and consider the allele values as coming from a continuous rather than a discrete distribution, so the forms of mutation described above are no longer applicable. Instead it is common to change the allele value of each gene randomly within its domain given by a lower L_i and upper U_i bound,⁴ resulting in the following transformation:

$$\langle x_1, \dots, x_n \rangle \rightarrow \langle x'_1, \dots, x'_n \rangle, \quad \text{where } x_i, x'_i \in [L_i, U_i].$$

As with integer representations, two types can be distinguished according to the probability distribution from which the new gene values are drawn: uniform and nonuniform mutation.

⁴ We assume here that the domain of each variable is a single interval $[L_i, U_i] \subseteq \mathbb{R}$. The generalisation to a union of disjoint intervals is straightforward.

Uniform Mutation For this operator the values of x'_i are drawn uniformly randomly from $[L_i, U_i]$. This is the most straightforward option, analogous to bit-flipping for binary encodings and the random resetting for integer encodings. It is normally used with a positionwise mutation probability.

Nonuniform Mutation Perhaps the most common form of nonuniform mutation used with floating-point representations takes a form analogous to the creep mutation for integers. It is designed so that usually, but not always, the amount of change introduced is small. This is achieved by adding to the current gene value an amount drawn randomly from a Gaussian distribution with mean zero and user-specified standard deviation, and then curtailing the resulting value to the range $[L_i, U_i]$ if necessary. This distribution, shown in Eq. 4.1, has the feature that the probability of drawing a random number with any given magnitude is a rapidly decreasing function of the standard deviation σ . Approximately two thirds of the samples drawn will lie within plus or minus one standard deviation, which means that most of the changes made will be small, but there is nonzero probability of generating very large changes since the tail of the distribution never reaches zero. Thus the σ value is a parameter of the algorithm that determines the extent to which given values x_i are perturbed by the mutation operator. For this reason σ is often called the **mutation step size**. It is normal practice to apply this operator with probability one per gene, and instead the mutation parameter is used to control the standard deviation of the Gaussian and hence the probability distribution of the step sizes taken.

$$p(\Delta x_i) = \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{(\Delta x_i - \xi)^2}{2\sigma^2}}. \quad (4.1)$$

An alternative to the Gaussian distribution is the use of a Cauchy distribution, which has a ‘fatter’ tail. That is, the probabilities of generating larger values are slightly higher than for a Gaussian with the same standard deviation [469].

4.4.2 Self-adaptive Mutation for Real-Valued Representation

As described above, non-uniform mutation applied to continuous variables is usually done by adding some random variables from a Gaussian distribution, with zero mean and a standard deviation which controls the mutation step size. The concept of **self-adaptation** represents a solution to the problem of how to adapt the step-sizes, which has been successfully demonstrated in many domains, not only for real-valued, but also for binary and integer search spaces [24]. The essential feature is that the step sizes are also included in the chromosomes and they themselves undergo variation and selection.

Details on how to mutate the value of σ are given below. The key concept is that the mutation step sizes are not set by the user; rather the σ coevolves with the solutions (the $\bar{x}$ part). In order to achieve this behaviour it is essential

Over recent decades much experience has been gained over self-adaptation in Evolutionary Algorithms, in particular in Evolution Strategies. The accumulated knowledge has identified necessary conditions for self-adaptation:

1. $\mu > 1$ so that different strategies are present
2. generation of an offspring surplus: $\lambda > \mu$
3. a not too strong selective pressure (heuristic: $\lambda/\mu = 7$, e.g., (15,100))
4. (μ, λ) -selection (to guarantee extinction of misadapted individuals)
5. recombination, usually intermediate, of strategy parameters

4.4.3 Recombination Operators for Real-Valued Representation

In general, we have three options for recombining two floating-point strings. First, using an analogous operator to those used for bit-strings, but now split between floats. In other words, an allele is one floating-point value instead of one bit. This has the disadvantage (shared with all of the recombination operators described above) that only mutation can insert new values into the population, since recombination only gives us new combinations of existing values. Recombination operators of this type for floating-point representations are known as **discrete recombination** and have the property that if we are creating an offspring z from parents x and y , then the allele value for gene i is given by $z_i = x_i$ or y_i with equal likelihood.

Second, using an operator that, in each gene position, creates a new allele value in the offspring that lies between those of the parents. Using the terminology above, we have $z_i = \alpha x_i + (1 - \alpha)y_i$ for some α in $[0,1]$. In this way, recombination is now able to create new gene material, but it has the disadvantage that as a result of the averaging process the range of the allele values in the population for each gene is reduced. Operators of this type are known as **intermediate** or **arithmetic recombination**.

Third, using an operator that in each position creates a new allele value in the offspring which is close to that of one of the parents, but may lie outside them (i.e., bigger than the larger of the two values, or smaller than the lesser). Operators of this type can create new material without restricting the range. Operators of this type are known as **blend recombination**.

Three types of arithmetic recombination are described in [295]. In all of these, the choice of the parameter α is sometimes made at random over $[0,1]$, but in practice it is common to use a constant value, often 0.5 (in which case we have **uniform arithmetic recombination**).

Simple Arithmetic Recombination First pick a recombination point k . Then, for child 1, take the first k floats of parent 1 and put them into the child. The rest is the arithmetic average of parent 1 and 2:

$$\text{Child 1: } \langle x_1, \dots, x_k, \alpha \cdot y_{k+1} + (1 - \alpha) \cdot x_{k+1}, \dots, \alpha \cdot y_n + (1 - \alpha) \cdot x_n \rangle.$$

Child 2 is analogous, with x and y reversed (Fig. 4.8, top).

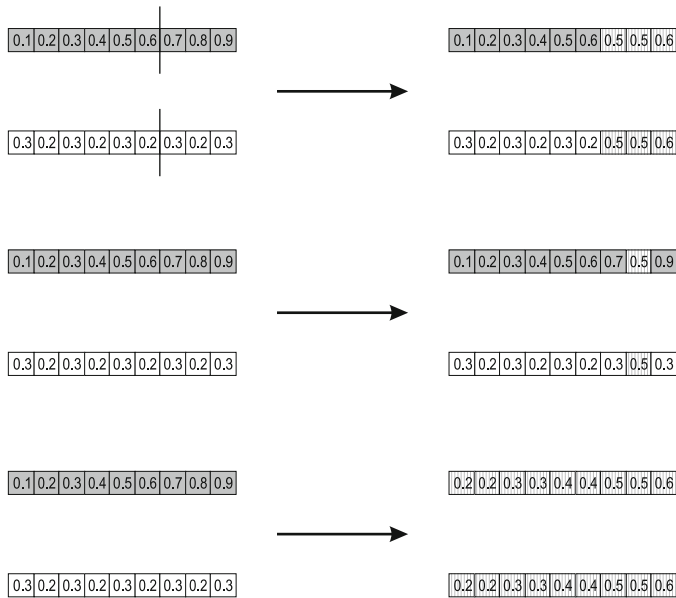


Fig. 4.8. Simple arithmetic recombination with $k=6, \alpha=1/2$ (top), single arithmetic recombination with $k=8, \alpha=1/2$ (middle), whole arithmetic recombination with $\alpha=1/2$ (bottom).

Single Arithmetic Recombination Pick a random allele k . At that position, take the arithmetic average of the two parents. The other points are the points from the parents, i.e.:

$$\text{Child 1: } \langle x_1, \dots, x_{k-1}, \alpha \cdot y_k + (1 - \alpha) \cdot x_k, x_{k+1}, \dots, x_n \rangle.$$

The second child is created in the same way with x and y reversed (Fig. 4.8, middle).

Whole Arithmetic Recombination This is the most commonly used operator and works by taking the weighted sum of the two parental alleles for each gene, i.e.:

$$\text{Child 1} = \alpha \cdot \bar{x} + (1 - \alpha) \cdot \bar{y}, \quad \text{Child 2} = \alpha \cdot \bar{y} + (1 - \alpha) \cdot \bar{x}.$$

This is illustrated in Fig. 4.8, bottom. As the example shows, if $\alpha=1/2$ the two offspring will be identical for this operator.

Blend Crossover Blend Crossover ($BLX-\alpha$) was introduced in [160] as a way of creating offspring in a region that is bigger than the (n-dimensional) rectangle spanned by the parents. The extra space is proportional to the

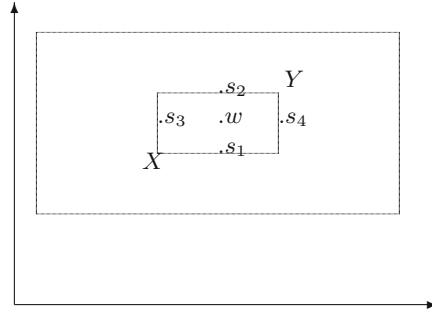


Fig. 4.9. Possible offspring from different recombination operators for two real-valued parents X and Y . $\{s_1, \dots, s_4\}$ are the four possible offspring from single arithmetic recombination with $\alpha = 0.5$. w is the offspring from whole arithmetic recombination with $\alpha = 0.5$ and the *inner* box represents all the possible offspring positions as α is varied. The *outer dashed* box shows all possible offspring positions for blend crossover with $\alpha = 0.5$ ($BLX - 0.5$), each position being equally likely.

distance between the parents and it varies per coordinate. If we have two parents x and y and assume that in position i the value $x_i < y_i$ then the difference $d_i = y_i - x_i$ and the range for the i th value in the child z is $[x_i - \alpha \cdot d_i, x_i + \alpha \cdot d_i]$. To create a child we can sample a random number u uniformly from $[0, 1]$, calculate $\gamma = (1 - 2\alpha)u - \alpha$, and set:

$$z_i = (1 - \gamma)x_i + \gamma y_i$$

Interestingly, the original authors reported best results with $\alpha = 0.5$, where the chosen values are equally likely to lie inside the two parent values as outside, so balancing exploration and exploitation.

Figure 4.9 illustrates the difference between single arithmetic recombination, whole arithmetic combination and Blend Crossover, with in each case the value of α set to 0.5. More recent methods such as Simulated Binary Crossover [111, 113] have built on Blend Crossover, so that rather than selecting offspring values uniformly from a range around each parent values, they are selected from a distribution which is more likely to create small changes, and the distribution is controlled by the distance between the parents.

4.5 Permutation Representation

Many problems naturally take the form of deciding on the order in which a sequence of events should occur. While other forms do occur (for example, decoder functions based on unrestricted integer representations [28, 201] or “floating keys” based on real-valued representations [27, 44]), the most natural representation of such problems is as a permutation of a fixed set of values

that can be represented as integers. One immediate consequence is that while a binary, or simple integer, representation allows numbers to occur more than once, such sequences of integers will not represent valid permutations. It is clear therefore that when choosing or designing variation operators to work with solutions that are represented as permutations, we require them to preserve the permutation property that each possible allele value occurs exactly once in the solution. We previously described one example, when we designed an EA for solving the N -queens problem efficiently, by representing each solution as a list of the rows on which each queen was positioned (with each on a different column), and insisted that these be a permutation so that no two queens shared the same row.

When choosing variation operators it is worth bearing in mind that there are actually two classes of problems that are represented by permutations. In the first of these, the *order* in which events occur is important. This might happen when the events use limited resources or time, and a typical example of this sort of problem is the production scheduling problem. This is the common problem of deciding in which order a series of times should be manufactured on a set of machines, where there may be dependencies between products, for example, there might be different set-up times between products, or one might be a component of another. As an example, it might be better for widget 1 to be produced before widgets 2 and 3, which in turn might be preferably produced before widget 4, no matter how far in advance this is done. In this case it might well be that the sequences [1,2,3,4] and [1,3,2,4] have similar fitness, and are much better than, for example, [4,3,2,1].

Another type of problem depends on *adjacency*, and is typified by the travelling salesperson problem (TSP). The problem is to find a complete tour of n given cities of minimal length. The search space for this problem is huge: there are $(n-1)!$ different routes possible for n given cities (for the asymmetric case counting back and forth as two routes).⁵ For $n = 30$ there are approximately 10^{32} different tours. Labelling the cities $1, 2, \dots, n$, a complete tour is a permutation, so that for $n = 4$, the routes [1,2,3,4] and [3,4,2,1] are both valid. The vital point here is that it is the links between cities that are important. The difference from order-based problems can clearly be seen if we consider that the starting point of the tour is also not important, thus [1,2,3,4], [2,3,4,1], [3,4,1,2], and [4,1,2,3] are all equivalent. Many examples of this class are also symmetric, so that [4,3,2,1] and so on are also equivalent.

Finally, we should mention that there are two possible ways to encode a permutation. In the first (most commonly used) of these the i th element of the representation denotes the event that happens in that place in the sequence (or the i th destination visited). In the second, the value of the i th element denotes the position in the sequence in which the i th event happens. Thus for the four cities [A,B,C,D], and the permutation [3,1,2,4], the first encoding denotes the tour [C,A,B,D] and the second [B,C,A,D].

⁵ These comments about problem size apply to all permutation problems.

4.5.1 Mutation for Permutation Representation

For permutation representations, it is no longer possible to consider each gene independently, rather finding legal mutations is a matter of moving alleles around in the genome. This has the immediate consequence that the mutation parameter is interpreted as the probability that the *chromosome* undergoes mutation, rather than that a single gene in the chromosome is altered. The three most common forms of mutation used for order-based problems were first described in [423]. Whereas the first three operators below (in particular insertion) work by making small changes to the order in which allele values occur, for adjacency-based problems these can cause huge numbers of links to be broken, and so inversion is more commonly used.

Swap Mutation Two positions (genes) in the chromosome are selected at random and their allele values swapped. This is illustrated in Fig. 4.10 (top), where the values in positions two and five have been swapped.

Insert Mutation Two alleles are selected at random and the second moved next to the first, shuffling along the others to make room. This is illustrated in Fig. 4.10 (middle), where the values two and five have been chosen.

Scramble Mutation Here the entire chromosome, or some randomly chosen subset of values within it, have their positions scrambled. This is illustrated in Fig. 4.10 (bottom), where the values from two to five have been chosen.

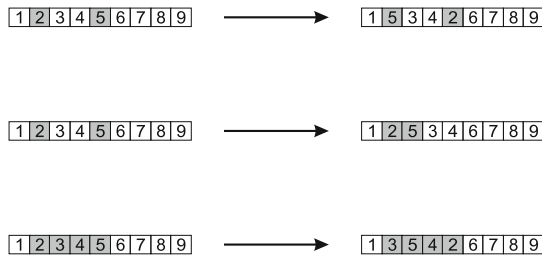


Fig. 4.10. Swap (top), insert (middle), and scramble mutation (bottom).

Inversion Mutation Inversion mutation works by randomly selecting two positions in the chromosome and reversing the order in which the values appear between those positions. It effectively breaks the chromosome into three parts, with all links inside a part being preserved, and only the two links between the parts being broken. The inversion of a randomly chosen substring is the thus smallest change that can be made to an adjacency-based problem, and all other changes can be easily constructed as a series of inversions. The

ordering of the search space induced by this operator thus forms a natural basis for considering this class of problems, equivalent to the Hamming space for binary problem representations. It is the basic move behind the 2-opt search heuristic for TSP [271], and by extension k -opt. This operator is illustrated in Fig. 4.11, where the substring between positions two and five was inverted.



Fig. 4.11. Inversion mutation

4.5.2 Recombination for Permutation Representation

At first sight, permutation-based representations present particular difficulties for the design of recombination operators, since it is not generally possible simply to exchange substrings between parents and still maintain the permutation property. However, this situation is alleviated when we consider what it is that the solutions actually represent, i.e., either an order in which elements occur, or a set of moves linking pairs of elements. A number of specialised recombination operators have been designed for permutations, which aim at transmitting as much as possible of the information contained in the parents, especially that held in common. We shall concentrate here on describing two of the best known and most commonly used operators for each subclass of permutation problems.

Partially Mapped Crossover (PMX) was first proposed by Goldberg and Lingle as a recombination operator for the TSP in [192], and has become one of the most widely used operators for adjacency-type problems. Over the years many slight variations of PMX appeared in the literature; here we use Whitley's definition from [452], which works as follows (Figs. 4.12–4.14).

1. Choose two crossover points at random, and copy the segment between them from the first parent (P1) into the first offspring.
2. Starting from the first crossover point look for elements in that segment of the second parent (P2) that have not been copied.
3. For each of these (say i), look in the offspring to see what element (say j) has been copied in its place from P1.
4. Place i into the position occupied by j in P2, since we know that we will not be putting j there (as we already have it in our string).
5. If the place occupied by j in P2 has already been filled in the offspring by an element k , put i in the position occupied by k in P2.

of any recombination operator is that of *respect*, i.e., that any information carried in both parents should also be present in the offspring. A moment's reflection tells us that this is clearly true for all of the recombination operators described above for binary and integer representations, and for discrete recombination for floating-point representations, but as the example above shows, is not necessarily true of PMX. With this issue in mind, several other operators have been designed for adjacency-based permutation problems, of which the best known is described next.

Edge crossover is based on the idea that offspring should be created as far as possible using only edges that are present in (one of) the parents. It has undergone a number of revisions over the years. Here we describe the most commonly used version: edge-3 crossover after Whitley [452], which is designed to ensure that common edges are preserved.

In order to achieve this, an edge table (also known as an adjacency list) is constructed, which for each element lists the other elements that are linked to it in the two parents. A '+' in the table indicates that the edge is present in both parents. The operator works as follows:

1. Construct the edge table
2. Pick an initial element at random and put it in the offspring
3. Set the variable *current_element* = *entry*
4. Remove all references to *current_element* from the table
5. Examine list for *current_element*
 - If there is a common edge, pick that to be the next element
 - Otherwise pick the entry in the list which itself has the shortest list
 - Ties are split at random
6. In the case of reaching an empty list, the other end of the offspring is examined for extension; otherwise a new element is chosen at random

Clearly only in the last case will so-called foreign edges be introduced.

Edge-3 recombination is illustrated by the following example where the parents are the same two permutations used in the PMX example [1 2 3 4 5 6 7 8 9] and [9 3 7 8 2 6 5 1 4], giving the edge table seen in [Table 4.2](#) and the construction illustrated in [Table 4.3](#). Note that only one child per recombination is created by this operator.

Element	Edges	Element	Edges
1	2,5,4,9	6	2,5+,7
2	1,3,6,8	7	3,6,8+
3	2,4,7,9	8	2,7+, 9
4	1,3,5,9	9	1,3,4,8
5	1,4,6+		

Table 4.2. Edge crossover: example edge table

Choices	Element selected	Reason	Partial result
All	1	Random	[1]
2,5,4,9	5	Shortest list	[1 5]
4,6	6	Common edge	[1 5 6]
2,7	2	Random choice (both have two items in list)	[1 5 6 2]
3,8	8	Shortest list	[1 5 6 2 8]
7,9	7	Common edge	[1 5 6 2 8 7]
3	3	Only item in list	[1 5 6 2 8 7 3]
4,9	9	Random choice	[1 5 6 2 8 7 3 9]
4	4	Last element	[1 5 6 2 8 7 3 9 4]

Table 4.3. Edge crossover: example of permutation construction

Order crossover This operator was designed by Davis for order-based permutation problems [98]. It begins in a similar fashion to PMX, by copying a randomly chosen segment of the first parent into the offspring. However, it proceeds differently because the intention is to transmit information about *relative order* from the second parent.

1. Choose two crossover points at random, and copy the segment between them from the first parent (P1) into the first offspring.
2. Starting from the second crossover point in the second parent, copy the remaining unused numbers into the first child in the order that they appear in the second parent, wrapping around at the end of the list.
3. Create the second offspring in an analogous manner, with the parent roles reversed.

This is illustrated in [Figs. 4.15](#) and [4.16](#).

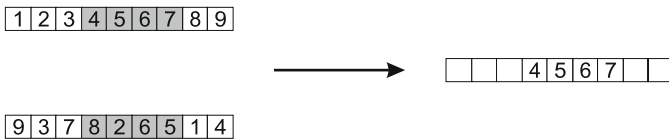


Fig. 4.15. Order crossover, step 1: copy randomly selected segment from first parent into offspring

Cycle Crossover The final operator that we will consider in this section is cycle crossover [325], which is concerned with preserving as much information as possible about the *absolute* position in which elements occur. The operator

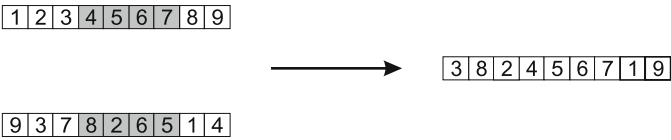


Fig. 4.16. Order crossover, step 2: copy rest of alleles in order they appear in second parent, treating string as toroidal

works by dividing the elements into *cycles*. A cycle is a subset of elements that has the property that each element always occurs paired with another element of the same cycle when the two parents are aligned. Having divided the permutation into cycles, the offspring are created by selecting alternate cycles from each parent. The procedure for constructing cycles is as follows:

1. Start with the first unused position and allele of P1
2. Look at the allele in the *same position* in P2
3. Go to the position with the *same allele* in P1
4. Add this allele to the cycle
5. Repeat steps 2 through 4 until you arrive at the first allele of P1

The complete operation of the operator is illustrated in [Fig. 4.17](#).

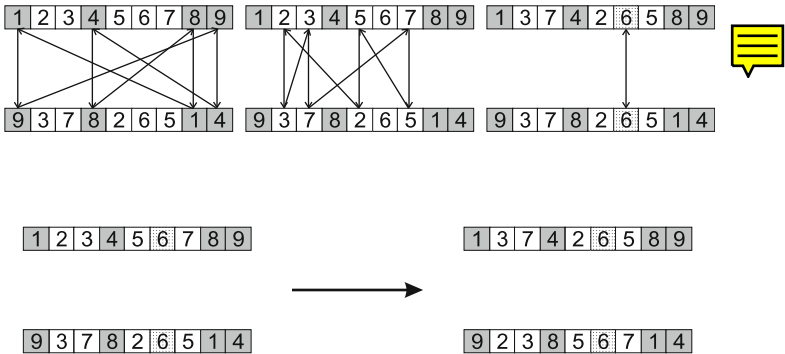


Fig. 4.17. Cycle crossover. Top: step 1- identification of cycles. Bottom: step 2- construction of offspring